

GEM5 USER GUIDE

Gem5 is an open-source computer architecture simulator that allows researchers, developers, and educators to study and analyze the performance, power, and other aspects of computer systems.



Overview of Gem5:

Gem5 is a cycle-accurate full-system computer architecture simulator. It is an open-source project that provides infrastructure for modeling multi-core architectures, including CPUs, memory systems, interconnects, caches, and other components of modern computer systems.

Gem5 allows researchers and developers to experiment with various hardware configurations, explore different architectural designs, and evaluate the performance of software running on simulated hardware. It is widely used in academia and industry for computer architecture research, performance analysis, and software development.

Gem5 supports various CPU models, including ARM, x86, RISC-V, and others, and it can simulate both single-core and multi-core systems. It provides detailed statistics and trace outputs that enable in-depth analysis of system behavior and performance.

Overall, gem5 is a powerful tool for computer architecture research, enabling the simulation of complex hardware systems and the evaluation of software workloads on those systems in a flexible and customizable environment.

How to install and build gem5 for windows

After setting up Ubuntu, we can proceed to install and build gem5 on Ubuntu within the Windows Subsystem for Linux (WSL) using the following steps:

1. **Update your system:** Open the terminal and update your package lists and upgrade your system packages to their latest versions by running:
-

```
sudo apt update  
sudo apt upgrade
```

2. **Install necessary dependencies:** Gem5 requires several dependencies to be installed. Run the following command to install them:
-

```
sudo apt install build-essential git m4 scons zlib1g zlib1g-dev \
libprotobuf-dev protobuf-compiler libprotoc-dev libgoogle-perftools-dev \
python3-dev libboost-all-dev pkg-config python3-tk
```

The need of each dependency is described below:

- **git** : gem5 uses git for version control.
- **gcc**: gcc is used to compile gem5. **Version >=8 must be used**. We support up to gcc Version 12. **Note**: GCC Version 9 may be used but is not officially supported due to it [increasing gem5 Object File sizes](#).
- **Clang**: Clang can also be used. At present, we support Clang 7 to Clang 16 (inclusive).
- **SCons** : gem5 uses SCons as its build environment. SCons 3.0 or greater must be used.
- **Python 3.6+** : gem5 relies on Python development libraries. gem5 can be compiled and run in environments using Python 3.6+.
- **protobuf 2.1+** (Optional): The protobuf library is used for trace generation and playback.
- **Boost** (Optional): The Boost library is a set of general purpose C++ libraries. It is a necessary dependency if you wish to use the SystemC implementation.

3. **Clone gem5 repository**: Use git to clone the gem5 repository:

```
git clone https://gem5.googlesource.com/public/gem5
```

4. **Navigate to the gem5 directory**: Change into the gem5 directory that you just cloned:

```
cd gem5
```

Before building Gem5 using SCons, increase the swap disk size on an Ubuntu system to prevent computer crashes caused by memory exhaustion. Increasing the swap size provides additional virtual memory, allowing the system to handle more tasks and applications without running out of memory, thereby improving stability and performance. In case you are not facing this error, skip this and proceed to step 14 directly.

This can be done by the following:

5. **Check Current Swap Usage:** Before creating a new swap file, check the current swap usage and available space using the **free** command:
-

```
free -h
```

6. **Check current swap configuration:** This command will output a list of active swap devices and files, their sizes, used space, and other relevant details, allowing you to verify the current swap configuration on your Ubuntu system quickly and easily.
-

```
sudo swapon --show
```

7. **Create a Swap file or Partition:** Decide whether you want to create a new swap file or resize an existing one.
-

- Create a new swap file- (for example, to create a 8GB swap file)

```
sudo fallocate -l 8G /swapfile
```

- Resizing an existing swap file- If you already have a swap file and want to resize it, skip this step and proceed to step 9.

8. **Set Permissions:** Set the appropriate permissions for the swap file.
-

```
sudo chmod 600 /swapfile
```

9. **Set Up Swap Area:** Make the file useable as swap by setting up the swap area using the new swap file.
-

```
sudo mkswap /swapfile
```

10. Enable the Swap File: Activate the newly created or resized swap file.

```
sudo swapon /swapfile
```

11. Verify the changes: Check that the new swap space is active and has the correct size.

```
sudo swapon --show
```

12. Make the Swap File Permanent: To make the swap file permanent across reboots, add it to the `/etc/fstab` file.

```
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

13. Re-Enable swap (if necessary): If you disabled swap earlier in the process, re-enable it now.

```
sudo swapon -a
```

That's it! Increasing swap disk size on your ubuntu system is successful.

14. Build gem5: Use scones to build gem5. The main scones file is called `SConstruct` and is found in the root of the source tree. Additional scones files are named `SConscript` and are found throughout the tree, usually near the files they're associated with.

Within the root of the gem5 directory, gem5 can be built with SCons using:

```
scons build/{ISA}/gem5.{variant} -j {cpus}
```

where `{ISA}` is the target (guest) Instruction Set Architecture, and `{variant}` specifies the compilation settings. For most intents and purposes `opt` is a good target for compilation. The `-j` flag is optional and allows for parallelization of compilation with `{cpus}` specifying the number of threads. A single-threaded compilation from scratch can take up to 2 hours on some systems. We therefore strongly advise allocating more threads if possible.

The valid ISAs are:

- ARM
- NULL
- MIPS
- POWER
- RISC-V
- SPARC
- X86

The valid build variants are:

- debug
- opt
- fast

For example, to build gem5 on 4 threads with `opt` and targeting x86:

```
scons build/X86/gem5.opt -j 4
```

Some common errors and fixes

- **Python package availability error:**

Package python is not available, but is referred to by another package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source

However the following packages replace it:

2to3 python2-minimal python2 dh-python python-is-python3

E: Package 'python-dev' has no installation candidate

E: Package 'python' has no installation candidate how to solve this
this error after i ran step 2

This might help:

1. Update package lists:

```
sudo apt update
```

2. Install Python 3 and related development packages:

```
sudo apt install python3 python3-dev
```

3. Install Python 2 if needed:

```
sudo apt install python2
```

- **Issue during the cloning process:**

```
scons: *** No SConstruct file found.  
File "/usr/lib/python3/dist-packages/SCons/Script/Main.py", line 947, in _main
```

This might help:

1. Navigate to the gem5 directory:

```
cd gem5
```

2. Verify the presence of SConstruct file:

```
ls
```

3. Re-clone gem5 repository:

```
git clone https://gem5.googlesource.com/public/gem5
```

- If the cloning process failed:

```
-bash: cd: gem5: No such file or directory
```

This might help:

1. Check the current directory:

```
pwd
```

2. Search for gem5 directory:

```
find / -type d -name "gem5" 2>/dev/null
```

3. Re-clone gem5 repository if needed:

```
git clone https://gem5.googlesource.com/public/gem5 /desired/location/gem5
```

4. Navigate to gem5 directory:

```
cd /path/to/gem5
```

- Python-related errors:

```
Package python is not available, but is referred to by another package.  
This may mean that the package is missing, has been obsoleted, or  
is only available from another source
```

```
However the following packages replace it:
```

```
2to3 python2-minimal python2 dh-python python-is-python3
```

```
E: Package 'python-dev' has no installation candidate
```

```
E: Package 'python' has no installation candidate
```

This might help:

1. Install Python 3:

```
sudo apt update  
sudo apt install python3
```

2. Install Python 3 development headers:

```
sudo apt install python3-dev
```

3. Reattempt the gem5 build:

```
cd /path/to/gem5  
scons build/X86/gem5.opt -j$(nproc) PYTHON=python3
```

Replace **/path/to/gem5** with the actual path to your gem5 directory.

- **Issues with the package names and dependencies:**

```
Package python is not available, but is referred to by another package.  
This may mean that the package is missing, has been obsoleted, or  
is only available from another source
```

```
However the following packages replace it:
```

```
2to3 python2-minimal python2 dh-python python-is-python3
```

```
E: Unable to locate package zlibg
```

```
E: Unable to locate package zlibg-dev
```

```
E: Unable to locate package libgoogle-pertools-dev
```

```
E: Package 'python-dev' has no installation candidate
```

```
E: Package 'python' has no installation candidate
```

This might help:

1. Python:

```
sudo apt update  
sudo apt install python3-dev
```

2. zlib and zlib development files:

```
sudo apt install zlib1g zlib1g-dev
```

3. libgoogle-perftools-dev:

```
sudo add-apt-repository universe  
sudo apt update
```

Then, install **libgoogle-perftools-dev**:

```
sudo apt install libgoogle-perftools-dev
```

- **Issue with the SCons configuration in your gem5 repository:**

```
You're missing the pre-commit/commit-msg hooks. These hook help to ensure your  
code follows gem5's style rules on git commit and your commit messages follow  
our commit message requirements. This script will now install these hooks in  
your .git/hooks/ directory.  
Press enter to continue, or ctrl-c to abort:  
Cannot find 'pre-commit'. Please ensure all Python requirements are  
installed. This can be done via 'pip install -r requirements.txt'.  
It is strongly recommended you install the pre-commit hooks before working with gem5. Do you want to  
continue compilation (y/n)?  
y  
  
TypeError: 'NoneType' object is not callable:  
File "/home/akshay/gem5/SConstruct", line 195:  
    main.Tool(SCons.Tool.FindTool(['gcc', 'clang'], main))  
File "/usr/lib/python3/dist-packages/SCons/Environment.py", line 1844:  
    tool(self)
```

This might help:

1. Install build tools:

```
sudo apt update  
sudo apt install build-essential
```

2. Install Python requirements:

```
pip install -r requirements.txt
```

- **M4, the macro processor, is missing:**

```
Error: Can't find version of M4 macro processor. Please install M4 and try again.
```

This might help:

1. Update the package lists to ensure you get the latest versions:

```
sudo apt update
```

2. Install M4:

```
sudo apt install m4
```

- **Issue during the compilation process:**

```
{standard input}: Assembler messages:
{standard input}:276927: Warning: end of file not at end of a line; newline inserted
{standard input}:278088: Error: invalid operands (*UND* and .gcc_except_table sections) for `-'
{standard input}:278110: Error: leb128 operand is an undefined symbol: .LE
g++: fatal error: Killed signal terminated program cc1plus
compilation terminated.
scons: *** [build/X86/arch/x86/linux/syscall_tbl64.o] Error 1
scons: building terminated because of errors.
```

This error message can be resolved by following the steps of changing the swap size mentioned in “How to install and build gem5”. (step-5 onwards)

Important directories of Gem5:

1. **Configs:**

The configs directory in gem5 contains various subdirectories, each serving a specific purpose in organizing configuration files for simulations. Here's a brief overview of each subdirectory:

1. **boot:**

- This directory contains configuration files related to booting simulations. It may include scripts and configurations for booting operating systems or bootloading custom firmware.

2. **common:**

- The common directory contains common configuration files that are used across different simulation scenarios. These files often define generic system configurations, CPU models, memory hierarchies, and input workloads that can be reused in multiple experiments.

3. **deprecated:**

- The deprecated directory contains deprecated or legacy configuration files that are no longer recommended for use. These files may include older versions of system configurations or simulation methodologies that have been superseded by newer alternatives.

4. **dist:**

- Configuration files related to distributed simulations are stored in the dist directory. These files define configurations for running gem5 simulations in a distributed computing environment, where multiple instances of gem5 collaborate to simulate a large-scale system.

5. **dram:**

- The dram directory contains configuration files specific to dynamic random-access memory (DRAM) subsystems. These files define parameters and settings for modeling DRAM components such as memory controllers, memory channels, and DRAM timings.

6. example:

- The example directory contains example configuration files demonstrating various gem5 features and simulation scenarios. These files serve as templates or starting points for creating custom simulations and experimenting with gem5 capabilities.

7. learning_gem5:

- This directory contains configuration files and examples related to the "Learning gem5" educational materials. It provides hands-on exercises and tutorials for learning gem5 simulation techniques and methodologies.

8. network:

- Configuration files related to network simulations are stored in the network directory. These files define network topologies, protocols, and parameters for simulating communication and networking scenarios using gem5.

9. nvm:

- The nvm directory contains configuration files specific to non-volatile memory (NVM) subsystems. These files define parameters and settings for modeling NVM technologies such as NAND flash, PCM, or MRAM.

10. ruby:

- Configuration files related to the Ruby memory system (Ruby is gem5's configurable memory system) are stored in the ruby directory. These files define configurations for modeling cache coherence protocols, directory controllers, and other components of the Ruby memory system.

11. splash2:

- Configuration files specific to the SPLASH-2 benchmark suite are stored in the splash2 directory. These files define configurations for running SPLASH-2 benchmarks and experiments within the gem5 simulation environment.

12. topologies:

- This directory contains configuration files for specifying system topologies, such as interconnect architectures, network-on-chip (NoC) designs, and cache hierarchies. These files define how components in the simulated system are interconnected and communicate with each other.
-

- Each subdirectory in the configs directory serves a specific purpose in organizing and categorizing configuration files for gem5 simulations, making it easier for users to locate and use relevant configurations for their experiments.

2.SRC:

Here's a brief overview of the directories and files in the gem5 source code repository (src directory) and their general purposes:

1. Doxyfile:

- This file is used for configuring the settings of Doxygen, a documentation generation tool. Doxygen is used to generate documentation from annotated source code.

2. Kconfig:

- Kconfig files are configuration files used by the Kconfig system, typically in the Linux kernel build process. They define configuration options and dependencies for the kernel.

3. SConscript:

- SConscript files are used by the SCons build system to specify how to build the source files in the directory.

4. arch:

- This directory contains source code related to architecture-specific components, such as instruction set architecture (ISA) definitions, architectural features, and CPU models.

5. base:

- The base directory contains fundamental components and utilities used throughout the gem5 source code, such as basic data structures, macros, and utility functions.

6. cpu:

- The cpu directory contains source code related to CPU models and processor simulation, including different CPU architectures, pipeline implementations, and microarchitectural features.

7. **dev:**

- The dev directory contains device models and implementations for various peripheral devices, such as disks, network interfaces, and input/output (I/O) devices.

8. **doc:**

- The doc directory contains documentation files, including user guides, developer documentation, and other resources related to gem5.

9. **doxygen:**

- This directory may contain additional files related to Doxygen documentation generation, such as configuration files or templates.

10. **gpu-compute:**

- The gpu-compute directory contains source code related to GPU (Graphics Processing Unit) simulation and compute kernels.

11. **kern:**

- The kern directory contains source code related to kernel-level components, such as operating system abstractions, system calls, and kernel interfaces.

12. **learning_gem5:**

- The learning_gem5 directory contains educational materials, tutorials, and example code for learning gem5 simulation techniques.

13. **mem:**

- The mem directory contains source code related to memory subsystem simulation, including memory controllers, caches, and memory models.

14. **proto:**

- The proto directory may contain protocol definitions or interface specifications used within gem5.

15. **python:**

- The python directory contains Python scripts and modules used for gem5 simulation configuration, scripting, and automation.

16. **sim:**

- The sim directory contains source code related to the gem5 simulation engine, including the main simulator loop, event scheduling, and simulation control logic.

17. **sst:**

- The sst directory contains source code related to SystemC simulation integration, allowing gem5 to interact with SystemC-based models.

18. **systemc:**

- The systemc directory contains SystemC wrapper code and integration components for incorporating SystemC models into gem5 simulations.

These directories and files collectively form the gem5 source code repository and contain the source code and resources necessary for building and using the gem5 simulator.

Simulation Workflows

- **Simulation Modes:** Explain the different simulation modes supported by gem5, such as syscall emulation, system call-based simulation, SE mode, FS mode, and Full-System mode.
- **Running Simulations:** Provide instructions for setting up and running simulations using gem5, including specifying simulation parameters, configuring CPU models, memory hierarchy, input workloads, and other simulation options.
- It seems like you're referring to two important scripts used in gem5 simulations: ``fs.py`` and ``se.py``. Let me explain what each of these scripts does:

Full System and Syscall Emulation :

1. **fs.py:**

- **Purpose:** ``fs.py`` stands for "Full System". This script is used to set up and run gem5 simulations in full-system mode, where gem5 simulates an entire computer system, including the operating system (e.g., Linux) and user applications.
- **Functionality:** The ``fs.py`` script configures gem5 to boot a guest operating system (such as a Linux distribution) and execute user-defined workloads or benchmarks within the simulated environment. It initializes the simulation with the necessary parameters, including CPU models, memory hierarchy, disk images, and other system components.

- **Usage:** Users typically use ``fs.py`` to run simulations that require a complete system environment, enabling them to study system-level behaviors, performance characteristics, and interactions between software and hardware components.

2. **se.py:**

- **Purpose:** ``se.py`` stands for "Syscall Emulation". This script is used to set up and run gem5 simulations in system call emulation mode, where gem5 emulates the behavior of a user-space application running on a specific operating system.

- **Functionality:** The ``se.py`` script allows users to execute user-space applications (e.g., binaries compiled for a specific architecture and operating system) within the gem5 simulation environment. It intercepts system calls made by the application and emulates their behavior, enabling the application to run as if it were executing on a real system.

- **Usage:** Users typically use ``se.py`` for running individual applications or workloads within a controlled environment. It is useful for studying application behavior, performance profiling, and debugging without the overhead of simulating the entire system.

Both ``fs.py`` and ``se.py`` are essential scripts in gem5 simulations, enabling users to conduct a wide range of experiments and studies in computer architecture, system-level performance analysis, and software development. Depending on the research objectives and requirements, users can choose the appropriate script and simulation mode to achieve their goals effectively.

Different Folders in Simulations

In gem5, different folders and files are used for organizing and managing simulations. Here's an overview of some common folders and files you may encounter when working with gem5 simulations:

1. **configs:**

- This folder contains configuration scripts and templates for setting up gem5 simulations. It includes subdirectories for various simulation configurations, such as CPU models, memory configurations, system components, and benchmark workloads.

2. src:

- The source code for gem5 is located in this folder. It includes the core simulation engine, CPU models, memory hierarchy components, device models, system-level components, scripting interfaces, and other essential modules.

3. build:

- This folder contains the build artifacts generated during the compilation of gem5 source code. It includes compiled binaries, object files, libraries, and other build-related files. The specific structure of the build folder may vary depending on the build configuration and options used.

4. benchmarks:

- This folder contains benchmark programs and workloads that can be used for performance evaluation and testing in gem5 simulations. It includes subdirectories for different benchmark suites, such as SPEC CPU, PARSEC, SPLASH-2, MiBench, and others.

5. configs/example:

- The `configs/example` folder contains example configuration scripts and templates for running gem5 simulations. These scripts serve as starting points for users to create their own simulation configurations or experiment setups.

6. system:

- This folder contains system-level components and models used in gem5 simulations, such as interconnect models, bus models, device controllers, and other system-level infrastructure.

7. util:

- The `util` folder contains utility scripts, tools, and resources used for gem5 development, testing, and management. It includes scripts for building gem5, running tests, generating documentation, and performing various administrative tasks.

8. tests:

- This folder contains test cases and test scripts used for testing gem5 functionality, verifying correctness, and detecting regressions. It includes unit tests, integration tests, regression tests, and other types of tests covering different aspects of gem5 functionality.
-

9. docs:

- The `docs` folder contains documentation files, including user guides, developer documentation, release notes, and other resources. It may include documentation in various formats, such as Markdown, HTML, PDF, and others.

10. configs/common:

- This folder contains common configuration scripts and templates shared across different gem5 simulation setups. It includes configuration files for common components, benchmarks, workloads, and system configurations used in gem5 simulations.

These folders and files play essential roles in organizing, managing, and conducting gem5 simulations, providing users with the necessary resources, tools, and infrastructure for building, configuring, running, and analyzing simulations effectively.

Different Benchmark suites available in Gem5

Gem5 supports various benchmark suites that users can utilize to evaluate and analyze the performance of simulated computer systems. Some of the commonly used benchmark suites available in gem5 include:

1. SPEC CPU:

- SPEC CPU benchmarks are widely used for evaluating the performance of CPUs. They consist of a suite of compute-intensive benchmarks that simulate real-world applications, including scientific computing, media encoding, and computational finance.

2. PARSEC:

- PARSEC (Parallel Applications for Shared Memory Computers) is a benchmark suite designed to evaluate the performance of parallel computing systems. It includes a diverse set of multi-threaded benchmarks representing different parallel programming paradigms and application domains.

3. SPLASH-2:

- SPLASH-2 is a benchmark suite developed by the University of California, Berkeley, to evaluate the performance of parallel computing systems. It includes a set of scientific computing applications, such as dense matrix multiplication, sparse matrix-vector multiplication, and FFT (Fast Fourier Transform).

4. SPECweb:

- SPECweb benchmarks are used to evaluate the performance of web servers and web-based applications. They simulate real-world web server workloads, including serving static and dynamic content, handling HTTP requests, and managing user sessions.

5. MiBench:

- MiBench is a benchmark suite designed to evaluate the performance of embedded systems and applications. It includes a collection of small, real-world programs representing typical tasks performed by embedded devices, such as automotive systems, consumer electronics, and IoT (Internet of Things) devices.

6. NetBench:

- NetBench benchmarks are used to evaluate the performance of network communication and networking protocols. They include applications and workloads representing various network traffic patterns, such as file transfer, video streaming, and web browsing.

7. Rodinia:

- Rodinia is a benchmark suite developed by the University of Virginia to evaluate the performance of heterogeneous computing systems, including CPUs and GPUs. It includes a variety of scientific computing and data-intensive applications, such as data mining, bioinformatics, and computational fluid dynamics.

8. SPEC Power:

- SPEC Power benchmarks are used to evaluate the power consumption and efficiency of computer systems. They simulate real-world workloads representing typical usage scenarios, such as web serving, database processing, and financial modeling.

These benchmark suites cover a wide range of application domains and workloads, allowing users to perform comprehensive performance analysis and evaluation of simulated computer systems in gem5. Depending on the research objectives and target system characteristics, users can select the appropriate benchmark suite or combination of benchmarks to conduct their experiments effectively.

First code:

Built the code while specifying its directory structure followed by the file name.

```
root@KRISHNA:~/gem5/configs/tutorial/part1# /root/gem5/build/X86/gem5.opt cutie.py
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 23.1.0.0
gem5 compiled Apr 15 2024 16:39:04
gem5 started Apr 21 2024 01:46:44
gem5 executing on KRISHNA, pid 5598
command line: /root/gem5/build/X86/gem5.opt cutie.py

heyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
root@KRISHNA:~/gem5/configs/tutorial/part1# |
```

No stats as no hardware specification are provided.

Working on caches:

1. **simple caches:** This file creates a barebones system and executes 'hello', a simple Hello World application. Adds a simple cache between the CPU and the membus.

- import the m5 (gem5) library created when gem5 is built
- import all of the SimObject
- create the system we are going to simulate
- Set the clock frequency of the system
- Create a simple CPU
- Create a memory bus, a coherent crossbar
- Create a simple cache
- Connect the I and D cache ports of the CPU to the memobj.
- Since cpu_side is a vector port, each time one of these is connected, it .create a new instance of the CPUSidePort class
- create the interrupt controller for the CPU and connect to the membus
- Create a process for a simple "Hello World" application
- set up the root SimObject and start the simulation

Output from stats.txt:

simSeconds	0.000053
>simTicks	53062000
>finalTick	53062000
>simFreq	1000000000000
>hostSeconds	0.03
>hostTickRate	1691219124
>hostMemory	630188
>simInsts	6192
>simOps	11149
>hostInstRate	196671
>hostOpRate	354038
>system.cache.hits	9088
>system.cache.misses	884
>system.cache.missLatency::samples	884
>system.cache.missLatency::mean	48744.343891
>system.cache.missLatency::gmean	41259.015839
>system.cache.missLatency::stdev	33722.608057

2. Two level cache:

```
----- Begin Simulation Statistics -----
simSeconds          0.000057          # Number of seconds simulated (Second)
simTicks            57467000          # Number of ticks simulated (Tick)
finalTick           57467000          # Number of ticks from beginning of simulation (restored
from checkpoints and n>simFreq        1000000000000          # The number of ticks per
simulated second ((Tick/Second))
hostSeconds         0.09              # Real time elapsed on the host (Second)
hostTickRate        667770573         # The number of ticks simulated per host second
(ticks/s) ((Tick/Second))
hostMemory          636968            # Number of bytes of host memory used (Byte)
simInsts            6192              # Number of instructions simulated (Count)
simOps              11149             # Number of ops (including micro ops) simulated (Count)
hostInstRate        71862             # Simulator instruction rate (inst/s) ((Count/Second))
hostOpRate          129379            # Simulator op (including micro ops) rate (op/s)
((Count/Second))
system.clk_domain.clock          1000          # Clock period in ticks (Tick)
system.clk_domain.voltage_domain.voltage  1          # Voltage in Volts (Volt)
system.cpu.numCycles             57467        # Number of cpu cycles simulated (Cycle)
system.cpu.cpi                   9.261402     # CPI: cycles per instruction (core level) ((Cycle/Count))
```

3. Ruby Cache:

"Ruby" refers to a flexible and highly configurable memory system model. The Ruby memory system is used to simulate various levels of cache coherence protocols, including MESI (Modified, Exclusive, Shared, Invalid) and MOESI (Modified, Owned, Exclusive, Shared, Invalid), which are commonly used in modern multicore processors.

- **Direct ruby test:** specific test or benchmark designed to evaluate and verify the functionality and performance of the Ruby memory system implementation directly, without running a full system simulation. This test is often used during the development and validation of the Ruby memory system within Gem5.

Output from stats.txt

```
----- Begin Simulation Statistics -----
simSeconds          0.000007          # Number of seconds simulated (Second)
simTicks            7399              # Number of ticks simulated (Tick)
finalTick           7399              # Number of ticks from beginning of simulation (restored
from checkpoints and n>simFreq        1000000000          # The number of ticks per
simulated second ((Tick/Second))
hostSeconds         0.02              # Real time elapsed on the host (Second)
hostTickRate        346801            # The number of ticks simulated per host second
(ticks/s) ((Tick/Second))
hostMemory          695340            # Number of bytes of host
memory used (Byte)
system.clk_domain.clock          1          # Clock period in ticks (Tick)
system.cpu.power_state.pwrStateResidencyTicks:
:UNDEFINED          7399              # Cumulative time (in ticks) in various power states (Tick)
system.mem_ctrls.avgPriority_ruby.dir_cntrl0::samples  100.00          # Average QoS priority value for
accepted requests (Count)
system.mem_ctrls.priorityMinLatency  0.000000019000          # per QoS priority minimum request to
response latency (Second)
system.mem_ctrls.priorityMaxLatency  0.000000033000          # per QoS priority maximum request to
response latency (Second)
```

- **Ruby random test:** The "Ruby Random Test" in Gem5 refers to a test designed to evaluate the behavior and performance of the Ruby memory system under random or stochastic workload scenarios.

Output from stats.txt

```
----- Begin Simulation Statistics -----
simSeconds          0.000275          # Number of seconds simulated (Second)
simTicks            275071            # Number of ticks simulated (Tick)
finalTick           275071            # Number of ticks from beginning of simulation
(restored from checkpoints and n>simFreq          1000000000          # The
number of ticks per simulated second ((Tick/Second))
hostSeconds         0.06              # Real time elapsed on the host (Second)
hostTickRate        4443438           # The number of ticks simulated per host
second (ticks/s) ((Tick/Second))
hostMemory          690472            # Number of bytes of host memory used (Byte)
system.clk_domain.clock 1            # Clock period in ticks (Tick)
```

4. DRAM:

Dynamic Random Access Memory (DRAM) is a type of volatile memory used in computers and other electronic devices for storing data that is being actively used or processed by the CPU. DRAM is volatile memory, meaning that it requires power to retain data. When power is removed, the data stored in DRAM is lost. This is in contrast to non-volatile memory types like ROM or flash memory. Like other types of RAM, DRAM is random access memory, which means that the CPU can access any location in memory with equal latency regardless of the memory's physical location. This makes it suitable for use as main memory in computers.

Output

```
hitu@DESKTOP-VUNOOPM:~/gem5/configs/example$ ls
apu_se.py      garnet_synth_traffic.py  hmc_test.py      memtest.py      ruby_gpu_random_test.py
arm            gem5_library             hsaTopology.py   noc_config      ruby_mem_test.py
dramsys.py     gpufs                    lupv              read_config.py   ruby_random_test.py
etrace_replay.py hmc_hello.py            m5out            riscv            sc_main.py
fs.py          hmc_tgen.cfg             memcheck.py      ruby_direct_test.py se.py
hitu@DESKTOP-VUNOOPM:~/gem5/configs/example$ cd ..
hitu@DESKTOP-VUNOOPM:~/gem5/configs$ ls
boot common deprecated dist dram example learning_gem5 m5out network nvm ruby splash2 t
hitu@DESKTOP-VUNOOPM:~/gem5/configs$ cd dram
hitu@DESKTOP-VUNOOPM:~/gem5/configs/dram$ ls
lat_mem_rd.py low_power_sweep.py sweep.py
hitu@DESKTOP-VUNOOPM:~/gem5/configs/dram$ /home/hitu/gem5/build/X86/gem5.opt sweep.py
gem5 Simulator System. https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 23.1.0.0
gem5 compiled Apr 20 2024 18:12:03
gem5 started Apr 20 2024 19:34:48
gem5 executing on DESKTOP-VUNOOPM, pid 92541
command line: /home/hitu/gem5/build/X86/gem5.opt sweep.py

Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (4096 Mbytes) does not match the address
bytes)
src/mem/physical.cc:94: warn: Not reserving swap space. May cause SIGSEGV on actual usage
src/sim/simulate.cc:199: info: Entering event queue @ 0. Starting simulation...
DRAM sweep with burst: 64, banks: 8, max stride: 512, request generation period: 5000
hitu@DESKTOP-VUNOOPM:~/gem5/configs/dram$
```

Working on memory traffic:

Memory traffic refers to the flow of data between the processor (CPU) and the memory subsystem in a computer system. This flow of data includes both read and write operations initiated by the processor to access data stored in memory. Memory traffic is a critical aspect of system performance, as it directly impacts the overall throughput and latency of memory accesses.

This script is used for running a traffic generator connected to a memory device. It supports linear and random accesses with a configurable amount of write traffic.

By default, this script runs with one channel (two pseudo channels) of HBM2 and this channel is driven with 32GiB/s of traffic for 1ms.

Output

```
----- Begin Simulation Statistics -----
simSeconds          0.001000          # Number of seconds simulated (Second)
simTicks            1000000000         # Number of ticks simulated (Tick)
finalTick           1000000000         # Number of ticks from beginning of simulation (restored
from checkpoints and n>simFreq         1000000000000000000 # The number of ticks per
simulated second ((Tick/Second))
hostSeconds         1.17               # Real time elapsed on the host (Second)
hostTickRate        852886466          # The number of ticks simulated per host second
(ticks/s) ((Tick/Second))
hostMemory          367568             # Number of bytes of host memory used (Byte)
board.clk_domain.clock 1000            # Clock period in ticks (Tick)
board.memory.mem_ctrl.priorityMinLatency 0.000000032000 # per QoS priority minimum request
to response latency (Second)
board.memory.mem_ctrl.priorityMaxLatency 0.000002533498 # per QoS priority maximum request
to response latency (Second)
board.memory.mem_ctrl.numReadWriteTurnArounds 2808      # Number of turnarounds from
READ to WRITE (Count)
board.memory.mem_ctrl.numWriteReadTurnArounds 2808      # Number of turnarounds from
WRITE to READ (Count)
board.memory.mem_ctrl.numStayReadState 666563          # Number of times bus staying in READ
state (Count)
board.memory.mem_ctrl.numStayWriteState 171487         # Number of times bus staying in WRITE
state (Count)
board.memory.mem_ctrl.readReqs 330868      # Number of read requests accepted (Count)
board.memory.mem_ctrl.writeReqs 86523      # Number of write requests accepted (Count)
board.memory.mem_ctrl.readBursts 661736    # Number of controller read bursts, including
those serviced by the write queue>board.memory.mem_ctrl.writeBursts 173046 # Number
of controller write bursts, including those merged in the write queue >board.memory.mem_ctrl.servicedByWrQ
0 # Number of controller read bursts serviced by the write queue (Count)
board.memory.mem_ctrl.mergedWrBursts 0      # Number of controller write bursts merged
with an existing one (Count)
board.memory.mem_ctrl.neitherReadNorWriteReqs 0        # Number of requests that are neither
read nor write (Count)
board.memory.mem_ctrl.avgRdQLen 83.19      # Average read queue length when
enqueueing ((Count/Tick))
board.memory.mem_ctrl.avgWrQLen 62.25     # Average write queue length when
enqueueing ((Count/Tick))
board.memory.mem_ctrl.numRdRetry 28268     # Number of times read queue was full
causing retry (Count)
```

Benchmark x86-npb:

The NAS Parallel Benchmarks (NPB) are a set of benchmark programs developed by the NASA Advanced Supercomputing (NAS) Division to evaluate the performance of parallel supercomputers. These benchmarks represent typical computational kernels and parallel applications commonly found in scientific and engineering workloads.

Script to run NAS parallel benchmarks with gem5. The script expects the benchmark program to run. The input is in the format <benchmark_prog>.<class>.x The system is fixed with 2 CPU cores, MESI Two Level system cache and 3 GB DDR4 memory. It uses the x86 board. This script will count the total number of instructions executed in the ROI. It also tracks how much wallclock and simulated time.

The simulation may fail in the case of `mg` with class C as it uses 3.3 GB of memory (more information is available at <https://arxiv.org/abs/2010.13216>).

We warn the user here.

The simulation will fail in the case of `ft` with class C. We warn the user here.

Here we setup the board. The X86Board allows for Full-System X86 simulations

Here we set the FS workload, i.e., npb benchmark program

After simulation has ended you may inspect

`m5out/system.pc.com\_1.device` to the stdout, if any.

After the system boots, we execute the benchmark program and wait till the ROI
`workbegin` annotation is reached (m5_work_begin()). We start collecting# the number of
committed instructions till ROI ends (marked by `workend`).

We then finish executing the rest of the benchmark.

The first exit_event ends with a `workbegin` cause. This means that the system started
successfully and the execution on the program started.

We have completed up to this step using KVM cpu. Now we switch to timing cpu for
detailed simulation.

Next, we need to check if the user passed a value for --ticks. If yes, then we limit out
execution to this number of ticks during the ROI. Otherwise, we simulate until the ROI ends.
By specifying a ROI, you can limit the simulation to the portion that is relevant for your
analysis, which can save time and resources.

```

command line: /root/gem5/build/X86/gem5.opt x86-npb-benchmarks.py --benchmark npb-bt-a --ticks 10000

info: Using default config
Running the simulation
Using KVM cpu
Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (16384 Mbytes) does not match the address range assigned (2048 Mbytes)
src/mem/dram_interface.cc:690: warn: DRAM device capacity (16384 Mbytes) does not match the address range assigned (2048 Mbytes)
src/sim/kernel_workload.cc:46: info: kernel located at: /root/.cache/gem5/x86-linux-kernel-4.19.83
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group. Legacy stat is deprecated.
0: board.pc.south_bridge.cmos.rtc: Real-time clock set to Sun Jan 1 00:00:00 2012
board.pc.com_1.device: Listening for connections on port 3456
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group. Legacy stat is deprecated.
src/dev/intel_8254_timer.cc:128: warn: Reading current count from inactive timer.
board.remote_gdb: Listening for connections on port 7000
src/cpu/kvm/base.cc:169: info: KVM: Coalesced MMIO disabled by config.
src/cpu/kvm/base.cc:169: info: KVM: Coalesced MMIO disabled by config.
src/sim/simulate.cc:199: info: Entering event queue @ 0. Starting simulation...
src/cpu/kvm/perfevent.cc:191: panic: PerfKvmCounter::attach failed (2)
Memory Usage: 3949720 KBytes
src/cpu/kvm/perfevent.cc:191: panic: PerfKvmCounter::attach failed (2)
Memory Usage: 3949720 KBytes
Program aborted at tick 0
Aborted

```

Implementing code using C:

Basic code

```

Open +
1 #include <stdio.h>
2
3 int main() {
4     int sum = 0;
5     for (int i = 1; i <= 10000; i++){
6         sum += i;
7     }
8     printf("Sum of numbers from 1 to 1000 is: %d\n", sum);
9     return 0;
10 }
11

```

Compiled output

```

hitu@DESKTOP-VUNOOPM:~/gem5/configs$ ls
benchmark.c boot common deprecated dist dram example learning_gem5 m5out network nvm ruby splash2 topologies
hitu@DESKTOP-VUNOOPM:~/gem5/configs$ gcc -o benchmark benchmark.c
hitu@DESKTOP-VUNOOPM:~/gem5/configs$ ls
benchmark benchmark.c boot common deprecated dist dram example learning_gem5 m5out network nvm ruby splash2 topologies
hitu@DESKTOP-VUNOOPM:~/gem5/configs$

```

No cache output

```
1|
2|----- Begin Simulation Statistics -----
3|simSeconds          0.016269          # Number of seconds simulated (Second)
4|simTicks            16269312000        # Number of ticks simulated (Tick)
5|finalTick           16269312000        # Number of ticks from beginning of simulation (restored from checkpoints and never reset) (Tick)
6|simFreq             1000000000000      # The number of ticks per simulated second ((Tick/Second))
7|hostSeconds         1.63              # Real time elapsed on the host (Second)
8|hostTickRate        9978080202        # The number of ticks simulated per host second (ticks/s) ((Tick/Second))
9|hostMemory          677640            # Number of bytes of host memory used (Byte)
10|simInsts            149606            # Number of instructions simulated (Count)
11|simOps              322237            # Number of ops (including micro ops) simulated (Count)
12|hostInstRate        91739            # Simulator instruction rate (inst/s) ((Count/Second))
13|hostOpRate          197595            # Simulator op (including micro ops) rate (op/s) ((Count/Second))
14|system.clk_domain.clock 1000        # Clock period in ticks (Tick)
15|system.cpu.numCycles 32538624        # Number of cpu cycles simulated (Cycle)
16|system.cpu.cpi       217.395183       # CPI: cycles per instruction (core level) ((Cycle/Count))
17|system.cpu.ipc       0.004600        # IPC: instructions per cycle (core level) ((Count/Cycle))
```

L1 cache

```
1|
2|----- Begin Simulation Statistics -----
3|simSeconds          0.000483          # Number of seconds simulated (Second)
4|simTicks            483236000         # Number of ticks simulated (Tick)
5|finalTick           483236000         # Number of ticks from beginning of simulation (restored from checkpoints and never reset) (Tick)
6|simFreq             1000000000000     # The number of ticks per simulated second ((Tick/Second))
7|hostSeconds         0.64             # Real time elapsed on the host (Second)
8|hostTickRate        755241520        # The number of ticks simulated per host second (ticks/s) ((Tick/Second))
9|hostMemory          683784            # Number of bytes of host memory used (Byte)
10|simInsts            149606            # Number of instructions simulated (Count)
11|simOps              322237            # Number of ops (including micro ops) simulated (Count)
12|hostInstRate        233747            # Simulator instruction rate (inst/s) ((Count/Second))
13|hostOpRate          503459            # Simulator op (including micro ops) rate (op/s) ((Count/Second))
14|system.clk_domain.clock 1000        # Clock period in ticks (Tick)
15|system.cpu.numCycles 966472          # Number of cpu cycles simulated (Cycle)
16|system.cpu.cpi       6.457137        # CPI: cycles per instruction (core level) ((Cycle/Count))
17|system.cpu.ipc       0.154867        # IPC: instructions per cycle (core level) ((Count/Cycle))
```

L2 cache

```
1|
2|----- Begin Simulation Statistics -----
3|simSeconds          0.003395          # Number of seconds simulated (Second)
4|simTicks            3394502500        # Number of ticks simulated (Tick)
5|finalTick           3394502500        # Number of ticks from beginning of simulation (restored from checkpoints and never reset) (Tick)
6|simFreq             1000000000000     # The number of ticks per simulated second ((Tick/Second))
7|hostSeconds         0.90             # Real time elapsed on the host (Second)
8|hostTickRate        3786419653       # The number of ticks simulated per host second (ticks/s) ((Tick/Second))
9|hostMemory          686596            # Number of bytes of host memory used (Byte)
10|simInsts            149606            # Number of instructions simulated (Count)
11|simOps              322237            # Number of ops (including micro ops) simulated (Count)
12|hostInstRate        166042            # Simulator instruction rate (inst/s) ((Count/Second))
13|hostOpRate          359357            # Simulator op (including micro ops) rate (op/s) ((Count/Second))
14|system.clk_domain.clock 1000        # Clock period in ticks (Tick)
15|system.cpu.numCycles 6789005         # Number of cpu cycles simulated (Cycle)
16|system.cpu.cpi       45.350310        # CPI: cycles per instruction (core level) ((Cycle/Count))
17|system.cpu.ipc       0.022047        # IPC: instructions per cycle (core level) ((Count/Cycle))
```

L1 and L2 cache

```
1|
2|----- Begin Simulation Statistics -----
3|simSeconds          0.000527          # Number of seconds simulated (Second)
4|simTicks            526774500         # Number of ticks simulated (Tick)
5|finalTick           526774500         # Number of ticks from beginning of simulation (restored from checkpoints and never reset) (Tick)
6|simFreq             1000000000000     # The number of ticks per simulated second ((Tick/Second))
7|hostSeconds         0.64             # Real time elapsed on the host (Second)
8|hostTickRate        823995414        # The number of ticks simulated per host second (ticks/s) ((Tick/Second))
9|hostMemory          692740            # Number of bytes of host memory used (Byte)
10|simInsts            149606            # Number of instructions simulated (Count)
11|simOps              322237            # Number of ops (including micro ops) simulated (Count)
12|hostInstRate        233865            # Simulator instruction rate (inst/s) ((Count/Second))
13|hostOpRate          503707            # Simulator op (including micro ops) rate (op/s) ((Count/Second))
14|system.clk_domain.clock 1000        # Clock period in ticks (Tick)
15|system.cpu.numCycles 1053549         # Number of cpu cycles simulated (Cycle)
16|system.cpu.cpi       7.038911        # CPI: cycles per instruction (core level) ((Cycle/Count))
17|system.cpu.ipc       0.142867        # IPC: instructions per cycle (core level) ((Count/Cycle))
```